



BITS Pilani

Pilani | Dubai | Goa | Hyderabad

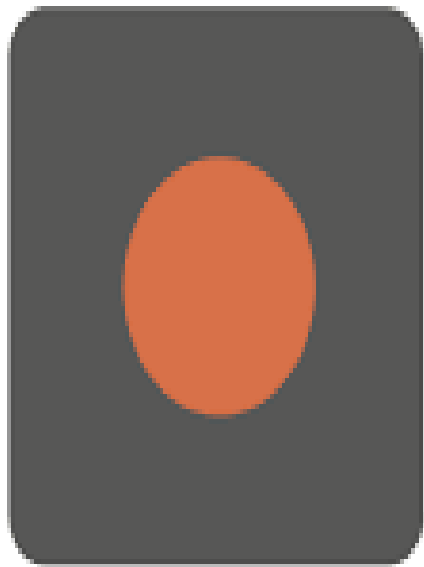
Compiler Design



Contact Session 6

LR(0) Parser Construction

IMP Note to Self



Start
Recording

IMP Note to Students

- It is important to know that just login to the session does not guarantee the attendance.
 - Once you join the session, continue till the end to consider you as present in the class.
 - IMPORTANTLY, you need to make the class more interactive by responding to Professors queries in the session.
 - **Whenever Professor calls your number / name ,you need to respond, otherwise it will be considered as ABSENT**
-

Objectives

- ✓ Bottom Up parsing
- ✓ Shift Reduce Parser
- ✓ LR Parser
- ✓ LR(0) Items and Automation
- ✓ LR(0) Parsing Process
- ✓ Conflicts in LR(0) Parsing

Bottom Up Parsing

- ✓ Bottom up parsing is the process of reducing (or **reduction** of) the input string of tokens into the start variable of the grammar.
- ✓ Bottom up parsing can be viewed as an attempt to **construct a parse tree** from the input string of terminals and moving upward to the root .
- ✓ **Goal** : Given an input string **w** and a grammar **G**, construct a parse tree by starting at the leaves and working to the root.

Working of Bottom Up Parsing

- ✓ Given a string of terminals as input, Read the input from left to right.
- ✓ Whenever we've matched the right hand side of a production, reduce it to the appropriate non-terminal and add that non-terminal to the parse tree.
- ✓ Build parse tree starting from leaves and working up toward the root.
- ✓ It is **Reverse of right-most derivation**.
- ✓ It is Used for type of grammars called LR Grammars.
 - ❖ L stands for leftmost scanning of the string.
 - ❖ R stands for Rightmost derivation in Reverse.

Example of Bottom Up Parsing

Given:

$$\begin{aligned} E &\rightarrow E + T \mid T \\ T &\rightarrow T * F \mid F \\ F &\rightarrow (E) \mid \text{id} \end{aligned}$$

and the string: **id * id**

id * id

$F * \text{id}$
|
id

$T * \text{id}$
|
 F
|
id

$T * F$
| |
 F **id**
|
id

T
/ | \
 T * F
| |
 F **id**
|
id

E
|
 T
/ | \
 T * F
| |
 F **id**
|
id

Reduction

- ✓ At each reduction step, a specific substring matching the body of a production is replaced by the non-terminal at the head of the production.
- ✓ But in derivation, a non-terminal in a sentential form is replaced by the body of one of its productions.
- ✓ So reduction is the reverse process of derivation.
- ✓ Reduction can be viewed as the Right Most Derivation in reverse order of the input string.
- ✓ Bottom up parser constructs a parse tree with ***Left to right (L)*** Scanning of the input string using **RMD in reverse (R)**.

Reduction

- ✓ Consider the following Grammar.

$$E = E + T \mid T$$

$$T \rightarrow T * F \mid F$$

$$F \rightarrow id$$

- ✓ Let the input string is $w = id * id$

Derivation Steps	Reduction Steps
$E \rightarrow T$ $\rightarrow T * F$ $\rightarrow T * id$ $\rightarrow F * id$ $\rightarrow id * id$	$id * id \rightarrow F * id$ $\rightarrow T * id$ $\rightarrow T * F$ $\rightarrow T$ $\rightarrow E$

Handle

- ✓ Informally, a “handle” is a substring that matches the body of a production (not necessarily the first one).

Grammar

$S \rightarrow a A B e$

$A \rightarrow A b c \mid b$

$B \rightarrow d$

a b b c d e

a A b c d e

a A d e

a A B e

S

Handle

String to parse

a b b c d e

a b b c d e

a A b c d e

a A A e

... ?

NOT a handle, because
further reductions will fail
(result is not a sentential form)

Handle Contd.

- ✓ **Handle Pruning:** Informally, a “handle” is a substring that matches the body of a production (not necessarily the first one), and reducing this handle, represents one step of reduction (or reverse rightmost derivation).
- ✓ Hence the process to construct a bottom-up parser is called handle-pruning.
- ✓ If G is unambiguous then every right-sentential form has a unique handle.

Right Sentential Form	Handle	Reducing Production
id * id	id	$F \rightarrow \text{id}$
F * id	F	$T \rightarrow F$
T * id	id	$F \rightarrow \text{id}$
T * F	T * F	$T \rightarrow T * F$
T	T	$E \rightarrow T$
E		

Shift Reduce Parser

- ✓ A shift reduce parser is a form of bottom up parser.
- ✓ It uses a stack to store the grammar symbols.
- ✓ It also uses an input buffer to hold the string to be parsed.
- ✓ We use **\$ symbol** to mark the bottom of the stack and also the right end of the input.
- ✓ Initially the stack is empty and the string w is in the input buffer.

Stack	Input Buffer
\$	w \$

Working of Shift Reduce Parser

- ✓ The handle is always present at the top of the stack.
- ✓ The parser operates by shifting zero or more input symbols onto the stack until a handle β is on the top of the stack.
- ✓ The parser then reduces the handle β to the left side of the appropriate production.
- ✓ The parser repeats this process until an error is found or until the stack contains the start state and the input string is empty.
- ✓ After entering this configuration, the parser halts and announces successful parsing.

Stack	Input Buffer
\$S	\$

Working of Shift Reduce Parser

- ✓ A shift-reduce parser has just four canonical actions:

Action	Description
Shift	Next input symbol is shifted onto the stack. This process is repeated until a handle appears on the top of the stack.
Reduce	When a handle is on top of stack, then it is reduced to the LHS of the appropriate production. i.e. pop handle off stack and push appropriate left end of handle the non-terminal in LHS of reducing production.
Accept	Terminate Parsing and Signal success
Error	Call an error recovery routine

Procedure

1. Initialize stack with \$ and the input buffer contains the string to be parsed followed by a \$ symbol at the end.
2. Repeat until the top of the stack is the '**start**' symbol and the input token is \$
 - a) **find the handle**: if we don't have a handle on top of the stack, *shift* an input symbol onto the stack.
 - b) **prune the handle** : if we have a handle β on the stack and $A \rightarrow \beta$ is the reducing production, *reduce by* $A \rightarrow \beta$ i.e.
 - i) pop β symbols off the stack
 - ii) push A onto the stack

Example

Considering the following grammar, parse the input string "id₁ * id₂" using a shift-reduce parser

$$E \rightarrow E + T \mid T$$

$$T \rightarrow T * F \mid F$$

$$F \rightarrow (E) \mid \text{id}$$

Stack	Input	Action
\$	id ₁ * id ₂ \$	Shift
\$id ₁	* id ₂ \$	Reduce by $F \rightarrow \text{id}$
\$F	* id ₂ \$	Reduce by $T \rightarrow F$
\$T	* id ₂ \$	Shift
\$T *	id ₂ \$	Shift
\$T * id ₂	\$	Reduce by $F \rightarrow \text{id}$
\$T * F	\$	Reduce by $T \rightarrow T * F$
\$T	\$	Reduce by $E \rightarrow T$
\$E	\$	Accept

Conflicts

- ✓ There are CFGs for which shift-reduce parsing can't be used.
- ✓ Conflicts arise in such cases during parsing.
- ✓ It is of two types.
 - Shift- reduce conflict
 - Reduce- reduce conflict
- ✓ Knowing the entire stack contents and the next input symbol, the parser can't decide whether to shift or to reduce.

Shift Reduce Conflicts

- ✓ Let $A \rightarrow B$ and $A \rightarrow Bab$ are the two productions.
- ✓ Let B be on the top of the stack and a is the current input symbol.
- ✓ This situation is known as shift-reduce conflict .
- ✓ Given the entire stack contents and the next input symbol, the parser can't decide whether to shift an input symbol or to reduce by some production at some situation..

Stack	Input Buffer	Action
\$B	<i>a</i> b\$	Shift a or Reduce $A \rightarrow B$

Reduce Reduce Conflicts

- ✓ Let $A \rightarrow a$ and $B \rightarrow a$ are the two productions.
- ✓ Let a be on the top of the stack.
- ✓ This situation is known as reduce-reduce conflict .
- ✓ When the parser has the option of reducing the handle with two different productions, then the parser can't decide which production to use at some situation.

Stack	Input Buffer	Action
\$ a	ba \$	Reduce $A \rightarrow a$ or Reduce $B \rightarrow a$

LR Parsers

- ✓ LR parsers present an efficient bottom up syntax analysis technique that can be used to parse a large class of context-free grammar.
- ✓ These techniques are called LR(k) parsing.
- ✓ L is for Left to right scanning of the input string.
- ✓ R is for constructing a rightmost derivation in reverse.
- ✓ K is for the number of input symbols of look ahead that are used in making parsing decision.
- ✓ When k is omitted, k is assumed to be 1.

Introduction Contd.

- ✓ LR parsing is attractive for the following reasons.
 - LR parsers can be constructed to recognize virtually all programming language constructs for which CFG can be written.
 - LR parsing method is the most general non-backtracking shift-reduce parsing method that can be implemented efficiently.
 - The LR parser can detect syntactic errors as soon as it is possible to do so on a left to right scan of the input string.
 - The class of grammars that can be parsed using LR methods is a proper superset of the class of grammars that can be parsed with predictive methods.

Introduction Contd.

- ✓ There are three common algorithms to build parsing tables for an “LR” parser:
- ✓ **Simple LR (SLR)**
 - smallest class of grammars and easiest to implement
 - smallest tables (number of states)
 - simple, fast construction
- ✓ **Canonical LR (CLR)**
 - full set of LR(1) grammars
 - largest tables (number of states)
 - slow, large construction
- ✓ **Look ahead LR (LALR)**
 - intermediate sized set of grammars
 - same number of states as SLR
 - canonical construction is slow and large
 - better construction techniques exist

Introduction Contd.

- ✓ CLR is more powerful than SLR and LALR
- ✓ LALR is more powerful than SLR
 - $\text{Power}(\text{CLR}) > \text{Power}(\text{LALR}) > \text{Power}(\text{SLR})$
- ✓ No. of states in CLR is more than SLR and LALR
- ✓ SLR and LALR has same number of states
 - $\text{States}(\text{CLR}) > \text{States}(\text{LALR}) \geq \text{States}(\text{SLR})$

LR(k) items

- ✓ The table construction algorithms use sets of LR(k) *items* or *configurations* to represent the possible states in a parse.
- ✓ An LR(k) item is a pair $[\alpha; \beta]$, where
 - α is a production from G with a ' . ' (dot) at some position in the RHS marking how much of the RHS of a production has already been seen.
 - β is a look ahead string containing k symbols (terminals or \$).
- ✓ Two cases of interest are $k = 0$ and $k = 1$:
 - LR(0) items play a key role in the SLR table construction algorithm.
 - LR(1) items play a key role in the CLR and LALR table construction algorithms.

LR(0) items

- ✓ In order to make shift-reduce decisions, the LR parser maintains some states to keep track of the position in a parsing process.
- ✓ Each state maintains some item.
- ✓ An **item** of a grammar G or **LR(0) item** is defined as a production of G with a dot at some position in the body.
- ✓ The • indicates how much of an item we have seen at a given state in the parse.

LR(0) items

- ✓ Considering a production $A \rightarrow XYZ$
 - $[A \rightarrow \bullet XYZ]$ indicates that the parser is looking for a string that can be derived from XYZ .
 - $[A \rightarrow XY \bullet Z]$ indicates that the parser has seen a string derived from XY and is looking for one derivable from Z
- ✓ Production $A \rightarrow XYZ$ generates 4 LR(0) items:
 - $A \rightarrow \bullet XYZ$
 - $A \rightarrow X \bullet YZ$
 - $A \rightarrow XY \bullet Z$
 - $A \rightarrow XYZ \bullet$
- ✓ Production $A \rightarrow \varepsilon$ has one item $[A \rightarrow \bullet]$

LR(0) Automation

- ✓ The collection of sets of LR(0) items is called canonical collection of LR(0) items.
- ✓ This collection provides the basis for constructing a DFA that is used to make parsing decisions. It is called LR(0) automation.
- ✓ It provides the basis for constructing a class of LR parsers called SLR.
- ✓ To construct a Canonical LR(0) collection for a grammar, we need to define:
 - Augmented grammar
 - Closure function
 - Go to function

LR(0) Automation Contd.

Augmented grammar:

- ✓ If G is a grammar with start symbol S , then G' is the augmented grammar for G with new start symbol S' and new production $S' \rightarrow S$
- ✓ If the grammar G is given by:

$$E \rightarrow E + T \mid T$$

$$T \rightarrow T * F \mid F$$

$$F \rightarrow (E) \mid \text{id}$$

It's augmented grammar G' will be:

$$E' \rightarrow E$$

$$E \rightarrow E + T \mid T$$

$$T \rightarrow T * F \mid F$$

$$F \rightarrow (E) \mid \text{id}$$

LR(0) Automation Contd.

Closure function:

- ✓ If I is a set of items for a grammar G , then $\text{closure}(I)$ is the set of items constructed from I .
- ✓ Create $\text{closure}(I)$ by the two rules:
 - Add every item in I to $\text{closure}(I)$
 - If $A \rightarrow \alpha \bullet B \beta$ is in $\text{closure}(I)$ and $B \rightarrow \gamma$ is a production, then add the item $B \rightarrow \bullet \gamma$ to $\text{closure}(I)$. Apply this rule until no more new items can be added to $\text{closure}(I)$.

LR(0) Automation Contd.

Procedure: **CLOSURE(I)**

repeat

if $[A \rightarrow \alpha \bullet B \beta]$ in I

add $[B \rightarrow \bullet \gamma]$ to I

until no more items can be added to I

return I

✓ Consider the following augmented grammar G'

$$\begin{aligned} E' &\rightarrow E \\ E &\rightarrow E + T \mid T \\ T &\rightarrow T * F \mid F \\ F &\rightarrow (E) \mid \text{id} \end{aligned}$$

Let $I = \{ E' \rightarrow \cdot E \}$, then

$$\begin{aligned} \text{closure}(I) = \{ & E' \rightarrow \bullet E \\ & E \rightarrow \bullet E + T \\ & E \rightarrow \bullet T \\ & T \rightarrow \bullet T * F \\ & T \rightarrow \bullet F \\ & F \rightarrow \bullet (E) \\ & F \rightarrow \bullet \text{id} \} \end{aligned}$$

LR(0) Automation Contd.

Goto function:

- ✓ If I is a set of items and X is a grammar symbol then $Goto(I, X)$ is defined to be the *closure* of the set of all items $[A \rightarrow \alpha X \bullet \beta]$ such that $[A \rightarrow \alpha \bullet X \beta]$ is in I .
- ✓ Goto function is used to define the transitions in the LR(0) automation for a grammar.

Procedure: GOTO(I, X)

let J be the set of items $[A \rightarrow \alpha X \bullet \beta]$

such that $[A \rightarrow \alpha \bullet X \beta]$ in I

return CLOSURE(J)

LR(0) Automation Contd.

Goto function:

- ✓ Considering the augmented grammar G'

Let $I = \{$
 $E' \rightarrow E \cdot$
 $E \rightarrow E \cdot + T \}$

then

$Goto(I, +) = \{$
 $E \rightarrow E + \cdot T$
 $T \rightarrow \cdot T * F$
 $T \rightarrow \cdot F$
 $F \rightarrow \cdot (E)$
 $F \rightarrow \cdot id \}$

Construction of LR(0) Automation

Algorithm:

Step-1: The grammar is augmented with a new start symbol S' and production $S' \rightarrow S$

Step-2: Initially, set $C = \text{closure}(\{[S' \rightarrow \bullet S]\})$ (this is the start state of the DFA)

Step-3: For each set of items $I \in C$ and each grammar symbol $X \in (N \cup T)$ such that $\text{GOTO}(I, X) \notin C$ and $\text{GOTO}(I, X) \neq \emptyset$, add the set of items $\text{GOTO}(I, X)$ to C

Step-4: Repeat 3 until no more sets can be added to C

Construction of LR(0) Automation

- ✓ Considering the grammar G

$$E \rightarrow E + T \mid T$$

$$T \rightarrow T * F \mid F$$

$$F \rightarrow (E) \mid \text{id}$$

- ✓ Its augmented grammar G' will be:

$$E' \rightarrow E$$

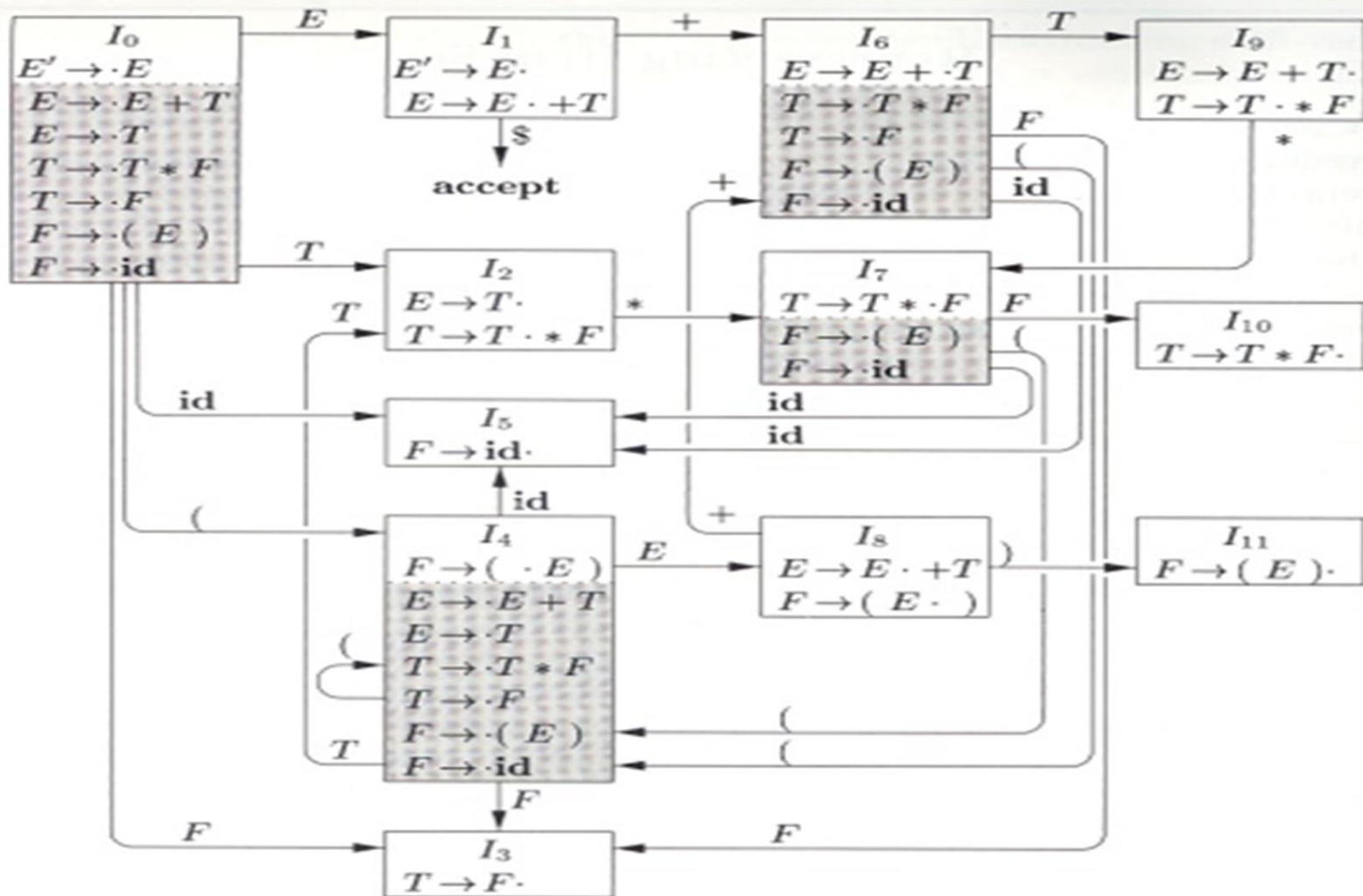
$$E \rightarrow E + T \mid T$$

$$T \rightarrow T * F \mid F$$

$$F \rightarrow (E) \mid \text{id}$$

$$\text{Now } I_0 = \text{closure}(\{E' \rightarrow \bullet E\}) = \left\{ \begin{array}{l} E' \rightarrow \bullet E \\ E \rightarrow \bullet E + T \\ E \rightarrow \bullet T \\ T \rightarrow \bullet T * F \\ T \rightarrow \bullet F \\ F \rightarrow \bullet (E) \\ F \rightarrow \bullet \text{id} \end{array} \right\}$$

Construction of LR(0) Automation



LR(0) Parsing Table

- ✓ The LR(0) Parsing table is constructed using the canonical collection of LR(0) items.
- ✓ This table is used in the SLR parsing process.
- ✓ **Structure:**
 - ✓ The parsing table has two type of entities.
 - A parsing-action function (ACTION)
 - A goto function (GOTO)

LR(0) Parsing Table contd.



✓ ACTION:

- The use of parsing table to make the shift-reduce decisions is the central idea of LR parsing.
- These decisions are entered in the action part of the parse table.

✓ GOTO:

- The goto part of the table shows only the state change as encountering non-terminals (except the non-terminal S' which we have added to make the grammar augmented).

LR(0) Parsing Table contd.



Steps to construct:

- ✓ Construct the collection of sets of LR(0) items for G
- ✓ No. of rows = No. of states
- ✓ No. of columns in ACTION section = No. of terminals including \$.
- ✓ No. of columns in GOTO section = No. of Non-terminals
- ✓ State i is constructed from I_i
 - If $[A \rightarrow \alpha \bullet a \beta]$ is in I_i and $\text{GOTO}(I_i, a) = I_j$ then $\text{ACTION}[i, a] \leftarrow \text{"shift } j\text{"}$
 - If $[A \rightarrow \alpha \bullet]$ is in I_i and $A \neq S'$ then $\text{ACTION}[i, a] \leftarrow \text{"reduce } A \rightarrow \alpha\text{"}$, for all a
 - If $[S' \rightarrow S \bullet]$ is in I_i then $\text{ACTION}[i, \$] \leftarrow \text{"accept"}$
- ✓ If $\text{GOTO}(I_i, A) = I_j$ then $\text{GOTO}[i, A] \leftarrow j$
- ✓ The undefined entries in ACTION and GOTO signifies "error"
- ✓ Initial state of parser is constructed from $\text{CLOSURE}([S0 \rightarrow \bullet S])$

Grammar:

$S \rightarrow E$

1. $E \rightarrow E + T$

2. $E \rightarrow T$

3. $T \rightarrow (E)$

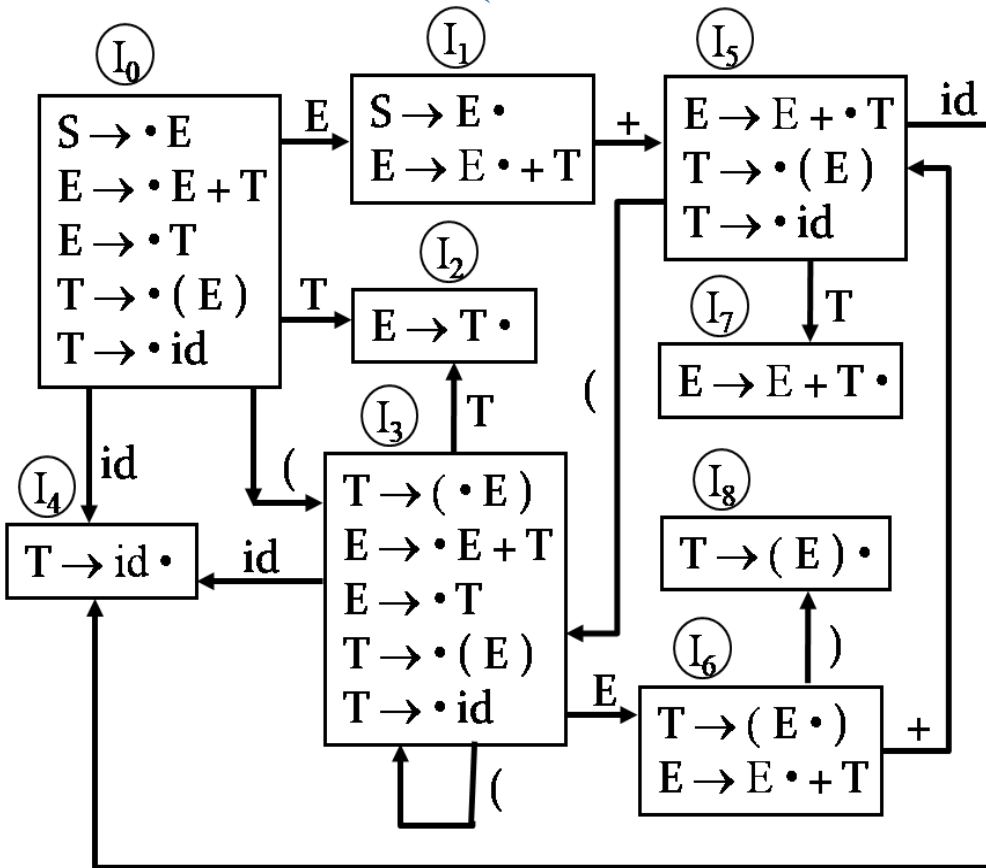
4. $T \rightarrow id$

LR(0) Collection

LR(0) Parse Table

action

goto



state	id	+	(	)	\$	S	E	T
0	s4		s3			1	2	
1		s5			acc			
2	r2	r2	r2	r2	r2			
3	s4		s3			6	2	
4	r4	r4	r4	r4	r4			
5	s4		s3					7
6		s5		s8				
7	r1	r1	r1	r1	r1			
8	r3	r3	r3	r3	r3			

shift & goto 5

reduce by
production #1

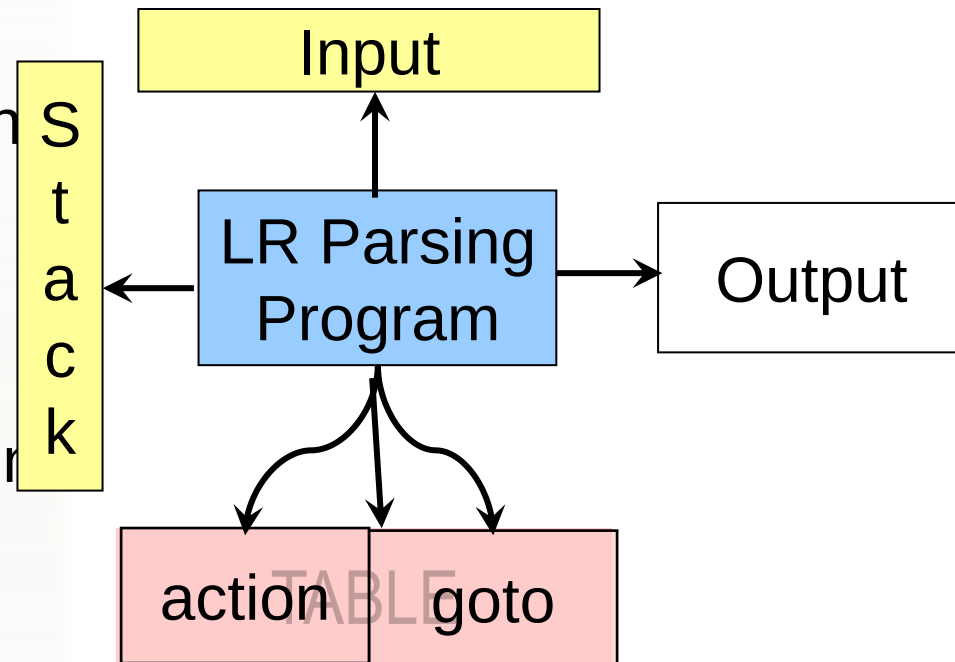
LR Parser



- Consists of : 1. A Driver Program 2. An Input 3. An Output
4. A stack and 5. A Parsing Table

The Stack

- Stores a string of the form $S_0 X_1 S_1 X_2 S_2 X_3 \dots X_m S_m$ with $(S_m$ at the top) where each:
 - 1. x_i is a grammar symbol &
 - 2. s_i is a symbol called **State** summarizing the information in the stack below it



- The parsing table has 2 parts
1. A parsing action function and a
 2. A goto function - goto

LR Parser Functioning

The parsing Program:

- Reads input characters (a) from input buffer one at a time
- Determines the current state on the top of the stack (s)
- Consults the Parsing table based on current state s and current input symbol a & gets $M[s, a]$ which can be either:
 - 1. Shift to state S
 - 2. Reduce by a grammar production
 - 3. Accept
 - 4. Error
- The function goto takes a state and grammar symbol as arguments and produces a new state
- The next move of parser is to read input a and state s and then consulting the entry action of parsing table
- If action $[s, a]$ is accept, parsing is complete

GRAMMAR:

- (1) $E \rightarrow E + T$
- (2) $E \rightarrow T$
- (3) $T \rightarrow T * F$
- (4) $T \rightarrow F$
- (5) $F \rightarrow (E)$
- (6) $F \rightarrow id$

LR Parser Example

OUTPUT:

INPUT:

id	+	id	*	id	\$
----	---	----	---	----	----

STACK:

0

LR Parsing
Program

State	action						goto		
	id	+	*	(	)	\$	E	T	F
0	s5			s4			1	2	3
1		s6				acc			
2		r2	s7		r2	r2			
3		r4	r4		r4	r4			
4	s5			s4			8	2	3
5		r6	r6		r6	r6			
6	s5			s4				9	3
7	s5			s4					10
8		s6			s11				
9		r1	s7		r1	r1			
10		r3	r3		r3	r3			
11		r5	r5		r5	r5			

GRAMMAR:

- (1) $E \rightarrow E + T$
- (2) $E \rightarrow T$
- (3) $T \rightarrow T * F$
- (4) $T \rightarrow F$
- (5) $F \rightarrow (E)$
- (6) $F \rightarrow id$

LR Parser Example

INPUT:

id	*	id	+	id	\$
----	---	----	---	----	----

LR Parsing Program

STACK:

5
id
0

State	action						goto		
	id	+	*	(	)	\$	E	T	F
0	s5			s4			1	2	3
1		s6				acc			
2		r2	s7		r2	r2			
3		r4	r4		r4	r4			
4	s5			s4			8	2	3
5		r6	r6		r6	r6			
6	s5			s4				9	3
7	s5			s4					10
8		s6			s11				
9		r1	s7		r1	r1			
10		r3	r3		r3	r3			
11		r5	r5		r5	r5			

OUTPUT:

F
|
id

GRAMMAR:

- (1) $E \rightarrow E + T$
- (2) $E \rightarrow T$
- (3) $T \rightarrow T * F$
- (4) $T \rightarrow F$
- (5) $F \rightarrow (E)$
- (6) $F \rightarrow id$

LR Parser Example

INPUT:

id	*	id	+	id	\$
----	---	----	---	----	----

STACK:

0

LR Parsing Program

OUTPUT:

F
|
id

State	action						goto		
	id	+	*	(	)	\$	E	T	F
0	s5			s4			1	2	3
1		s6				acc			
2		r2	s7		r2	r2			
3		r4	r4		r4	r4			
4	s5			s4			8	2	3
5		r6	r6		r6	r6			
6	s5			s4				9	3
7	s5			s4					10
8		s6			s11				
9		r1	s7		r1	r1			
10		r3	r3		r3	r3			
11		r5	r5		r5	r5			

GRAMMAR:

(1) $E \rightarrow E + T$

(2) $E \rightarrow T$

(3) $T \rightarrow T * F$

(4) $T \rightarrow F$

(5) $F \rightarrow (E)$

(6) $F \rightarrow \text{id}$

LR Parser Example

OUTPUT:

INPUT:

id * id + id \$

STACK:

3
F
0

LR Parsing
Program

State	action						goto		
	id	+	*	(	)	\$	E	T	F
0	s5			s4			1	2	3
1		s6				acc			
2		r2	s7		r2	r2			
3		r4	r4		r4	r4			
4	s5			s4			8	2	3
5		r6	r6		r6	r6			
6	s5			s4				9	3
7	s5			s4					10
8		s6			s11				
9		r1	s7		r1	r1			
10		r3	r3		r3	r3			
11		r5	r5		r5	r5			

T
|
F
|
id

GRAMMAR:

- (1) $E \rightarrow E + T$
- (2) $E \rightarrow T$
- (3) $T \rightarrow T * F$
- (4) $T \rightarrow F$
- (5) $F \rightarrow (E)$
- (6) $F \rightarrow id$

LR Parser Example

OUTPUT:

INPUT:

id	*	id	+	id	\$
----	---	----	---	----	----

STACK:

0

LR Parsing Program

State	action						goto		
	id	+	*	(	)	\$	E	T	F
0	s5			s4			1	2	3
1		s6				acc			
2		r2	s7		r2	r2			
3		r4	r4		r4	r4			
4	s5			s4			8	2	3
5		r6	r6		r6	r6			
6	s5			s4				9	3
7	s5			s4					10
8		s6			s11				
9		r1	s7		r1	r1			
10		r3	r3		r3	r3			
11		r5	r5		r5	r5			

T
|
F
|
id

GRAMMAR:

- (1) $E \rightarrow E + T$
- (2) $E \rightarrow T$
- (3) $T \rightarrow T * F$
- (4) $T \rightarrow F$
- (5) $F \rightarrow (E)$
- (6) $F \rightarrow id$

LR Parser Example

INPUT:

id	*	id	+	id	\$
----	---	----	---	----	----

LR Parsing Program

STACK:

2
T
0

OUTPUT:

T
|
F
|
id

State	action						goto		
	id	+	*	(	)	\$	E	T	F
0	s5			s4			1	2	3
1		s6				acc			
2		r2	s7		r2	r2			
3		r4	r4		r4	r4			
4	s5			s4			8	2	3
5		r6	r6		r6	r6			
6	s5			s4				9	3
7	s5			s4					10
8		s6			s11				
9		r1	s7		r1	r1			
10		r3	r3		r3	r3			
11		r5	r5		r5	r5			

GRAMMAR:

- (1) $E \rightarrow E + T$
- (2) $E' \rightarrow T$
- (3) $T \rightarrow T * F$
- (4) $T \rightarrow F$
- (5) $F \rightarrow (E)$
- (6) $F \rightarrow id$

LR Parser Example

INPUT:

id	*	id	+	id	\$
----	---	----	---	----	----

LR Parsing Program

STACK:

7
*
2
T
0

State	action						goto		
	id	+	*	(	)	\$	E	T	F
0	s5			s4			1	2	3
1		s6				acc			
2		r2	s7		r2	r2			
3		r4	r4		r4	r4			
4	s5			s4			8	2	3
5		r6	r6		r6	r6			
6	s5			s4				9	3
7	s5			s4					10
8		s6			s11				
9		r1	s7		r1	r1			
10		r3	r3		r3	r3			
11		r5	r5		r5	r5			

OUTPUT:

T
|
F
|
id

GRAMMAR:

- (1) $E \rightarrow E + T$
- (2) $E' \rightarrow T$
- (3) $T \rightarrow T * F$
- (4) $T \rightarrow F$
- (5) $F \rightarrow (E)$
- (6) $F \rightarrow id$

LR Parser Example

INPUT:

id	*	id	+	id	\$
----	---	----	---	----	----

LR Parsing Program

STACK:

5
id
7
*
2
T
0

State	action						goto		
	id	+	*	(	)	\$	E	T	F
0	s5			s4			1	2	3
1		s6				acc			
2		r2	s7		r2	r2			
3		r4	r4		r4	r4			
4	s5			s4			8	2	3
5		r6	r6		r6	r6			
6	s5			s4				9	3
7	s5			s4					10
8		s6			s11				
9		r1	s7		r1	r1			
10		r3	r3		r3	r3			
11		r5	r5		r5	r5			

OUTPUT:

T	F
F	id
id	

GRAMMAR:

- (1) $E \rightarrow E + T$
- (2) $E' \rightarrow T$
- (3) $T \rightarrow T * F$
- (4) $T \rightarrow F$
- (5) $F \rightarrow (E)$
- (6) $F \rightarrow id$

LR Parser Example

INPUT:

id	*	id	+	id	\$
----	---	----	---	----	----

OUTPUT:

STACK:

7
*
2
T
0

LR Parsing
Program

State	action						goto		
	id	+	*	(	)	\$	E	T	F
0	s5			s4			1	2	3
1		s6				acc			
2		r2	s7		r2	r2			
3		r4	r4		r4	r4			
4	s5			s4			8	2	3
5		r6	r6		r6	r6			
6	s5			s4				9	3
7	s5			s4					10
8		s6			s11				
9		r1	s7		r1	r1			
10		r3	r3		r3	r3			
11		r5	r5		r5	r5			

T		F
F		id
id		

GRAMMAR:

- (1) $E \rightarrow E + T$
- (2) $E' \rightarrow T$
- (3) $T \rightarrow T * F$
- (4) $T \rightarrow F$
- (5) $F \rightarrow (E)$
- (6) $F \rightarrow id$

LR Parser Example

INPUT:

id	*	id	+	id	\$
----	---	----	---	----	----

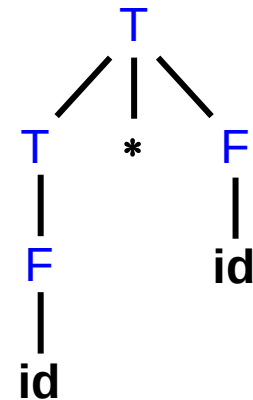
LR Parsing Program

STACK:

10
F
7
*
2
T
0

State	action						goto		
	id	+	*	(	)	\$	E	T	F
0	s5			s4			1	2	3
1		s6				acc			
2		r2	s7		r2	r2			
3		r4	r4		r4	r4			
4	s5			s4			8	2	3
5		r6	r6		r6	r6			
6	s5			s4				9	3
7	s5			s4					10
8		s6			s11				
9		r1	s7		r1	r1			
10		r3	r3		r3	r3			
11		r5	r5		r5	r5			

OUTPUT:



GRAMMAR:

(1) $E \rightarrow E + T$

(2) $E \rightarrow T$

(3) $T \rightarrow T * F$

(4) $T \rightarrow F$

(5) $F \rightarrow (E)$

(6) $F \rightarrow id$

LR Parser Example

INPUT:

id * id + id \$

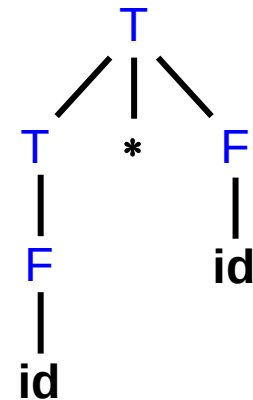
STACK:

10
F
7
*
2
T
0

LR Parsing
Program

State	action						goto		
	id	+	*	(	)	\$	E	T	F
0	s5			s4			1	2	3
1		s6				acc			
2		r2	s7		r2	r2			
3		r4	r4		r4	r4			
4	s5			s4			8	2	3
5		r6	r6		r6	r6			
6	s5			s4				9	3
7	s5			s4					10
8		s6			s11				
9		r1	s7		r1	r1			
10		r3	r3		r3	r3			
11		r5	r5		r5	r5			

OUTPUT:



GRAMMAR:

(1) $E \rightarrow E + T$

(2) $E \rightarrow T$

(3) $T \rightarrow T * F$

(4) $T \rightarrow F$

(5) $F \rightarrow (E)$

(6) $F \rightarrow id$

LR Parser Example

INPUT:

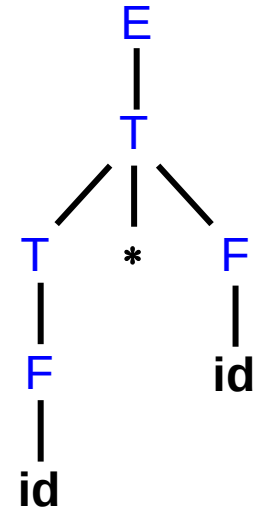
id	*	id	+	id	\$
----	---	----	---	----	----

STACK:

2
T
0

LR Parsing Program

OUTPUT:



State	action						goto		
	id	+	*	(	)	\$	E	T	F
0	s5			s4			1	2	3
1		s6				acc			
2		r2	s7		r2	r2			
3		r4	r4		r4	r4			
4	s5			s4			8	2	3
5		r6	r6		r6	r6			
6	s5			s4				9	3
7	s5			s4					10
8		s6			s11				
9		r1	s7		r1	r1			
10		r3	r3		r3	r3			
11		r5	r5		r5	r5			

GRAMMAR:

(1) $E \rightarrow E + T$

(2) $E \rightarrow T$

(3) $T \rightarrow T * F$

(4) $T \rightarrow F$

(5) $F \rightarrow (E)$

(6) $F \rightarrow id$

LR Parser Example

INPUT:

id	*	id	+	id	\$
----	---	----	---	----	----

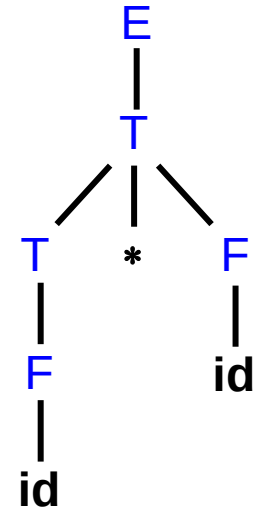
STACK:

0

LR Parsing Program

State	action						goto		
	id	+	*	(	)	\$	E	T	F
0	s5			s4			1	2	3
1		s6				acc			
2		r2	s7		r2	r2			
3		r4	r4		r4	r4			
4	s5			s4			8	2	3
5		r6	r6		r6	r6			
6	s5			s4				9	3
7	s5			s4					10
8		s6			s11				
9		r1	s7		r1	r1			
10		r3	r3		r3	r3			
11		r5	r5		r5	r5			

OUTPUT:



GRAMMAR:

- (1) $E \rightarrow E + T$
- (2) $E' \rightarrow T$
- (3) $T \rightarrow T * F$
- (4) $T \rightarrow F$
- (5) $F \rightarrow (E)$
- (6) $F \rightarrow id$

LR Parser Example

INPUT:

id	*	id	+	id	\$
----	---	----	---	----	----

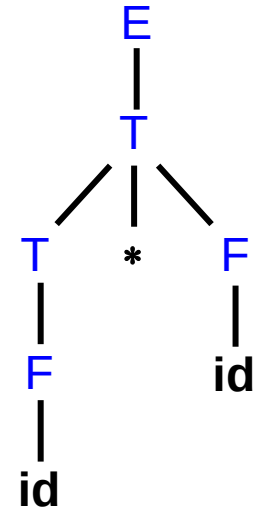
STACK:

1
E
0

LR Parsing Program

State	action						goto		
	id	+	*	(	)	\$	E	T	F
0	s5			s4			1	2	3
1		s6				acc			
2		r2	s7		r2	r2			
3		r4	r4		r4	r4			
4	s5			s4			8	2	3
5		r6	r6		r6	r6			
6	s5			s4				9	3
7	s5			s4					10
8		s6			s11				
9		r1	s7		r1	r1			
10		r3	r3		r3	r3			
11		r5	r5		r5	r5			

OUTPUT:



GRAMMAR:

- (1) $E \rightarrow E + T$
- (2) $E' \rightarrow T$
- (3) $T \rightarrow T * F$
- (4) $T \rightarrow F$
- (5) $F \rightarrow (E)$
- (6) $F \rightarrow id$

LR Parser Example

INPUT:

id	*	id	+	id	\$
----	---	----	---	----	----

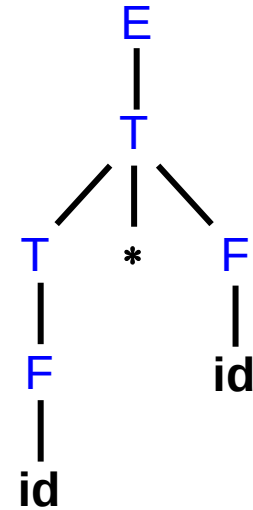
STACK:

6
+
1
E
0

LR Parsing Program

State	action						goto		
	id	+	*	(	)	\$	E	T	F
0	s5			s4			1	2	3
1		s6				acc			
2		r2	s7		r2	r2			
3		r4	r4		r4	r4			
4	s5			s4			8	2	3
5		r6	r6		r6	r6			
6	s5			s4				9	3
7	s5			s4					10
8		s6			s11				
9		r1	s7		r1	r1			
10		r3	r3		r3	r3			
11		r5	r5		r5	r5			

OUTPUT:



GRAMMAR:

- (1) $E \rightarrow E + T$
- (2) $E' \rightarrow T$
- (3) $T \rightarrow T * F$
- (4) $T \rightarrow F$
- (5) $F \rightarrow (E)$
- (6) $F \rightarrow id$

LR Parser Example

INPUT: id * id + id \$

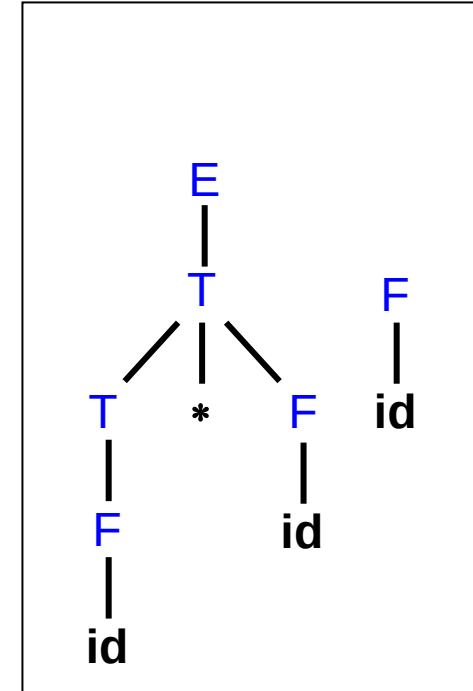
STACK:

5
id
6
+
1
E
0

LR Parsing Program

State	action						goto		
	id	+	*	(	)	\$	E	T	F
0	s5			s4			1	2	3
1		s6				acc			
2		r2	s7		r2	r2			
3		r4	r4		r4	r4			
4	s5			s4			8	2	3
5		r6	r6		r6	r6			
6	s5			s4				9	3
7	s5			s4					10
8		s6			s11				
9		r1	s7		r1	r1			
10		r3	r3		r3	r3			
11		r5	r5		r5	r5			

OUTPUT:



GRAMMAR:

- (1) $E \rightarrow E + T$
- (2) $E' \rightarrow T$
- (3) $T \rightarrow T * F$
- (4) $T \rightarrow F$
- (5) $F \rightarrow (E)$
- (6) $F \rightarrow id$

LR Parser Example

INPUT:

id	*	id	+	id	\$
----	---	----	---	----	----

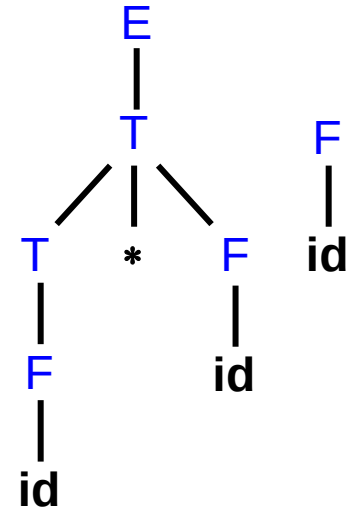
STACK:

6
+
1
E
0

LR Parsing Program

State	action						goto		
	id	+	*	(	)	\$	E	T	F
0	s5			s4			1	2	3
1		s6				acc			
2		r2	s7		r2	r2			
3		r4	r4		r4	r4			
4	s5			s4			8	2	3
5		r6	r6		r6	r6			
6	s5			s4				9	3
7	s5			s4					10
8		s6			s11				
9		r1	s7		r1	r1			
10		r3	r3		r3	r3			
11		r5	r5		r5	r5			

OUTPUT:



GRAMMAR:

- (1) $E \rightarrow E + T$
- (2) $E' \rightarrow T$
- (3) $T \rightarrow T * F$
- (4) $T \rightarrow F$
- (5) $F \rightarrow (E)$
- (6) $F \rightarrow id$

LR Parser Example

INPUT:

id	*	id	+	id	\$
----	---	----	---	----	----

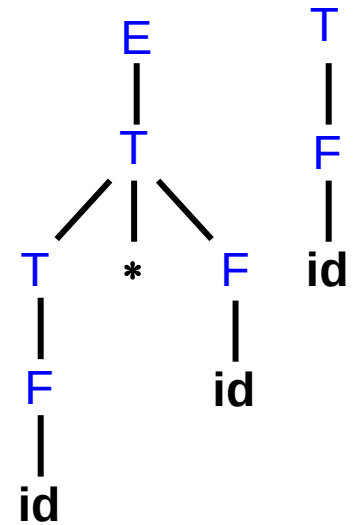
STACK:

3
F
6
+
1
E
0

LR Parsing Program

State	action						goto		
	id	+	*	(	)	\$	E	T	F
0	s5			s4			1	2	3
1		s6				acc			
2		r2	s7		r2	r2			
3		r4	r4		r4	r4			
4	s5			s4			8	2	3
5		r6	r6		r6	r6			
6	s5			s4				9	3
7	s5			s4					10
8		s6			s11				
9		r1	s7		r1	r1			
10		r3	r3		r3	r3			
11		r5	r5		r5	r5			

OUTPUT:



GRAMMAR:

- (1) $E \rightarrow E + T$
- (2) $E' \rightarrow T$
- (3) $T \rightarrow T * F$
- (4) $T \rightarrow F$
- (5) $F \rightarrow (E)$
- (6) $F \rightarrow id$

LR Parser Example

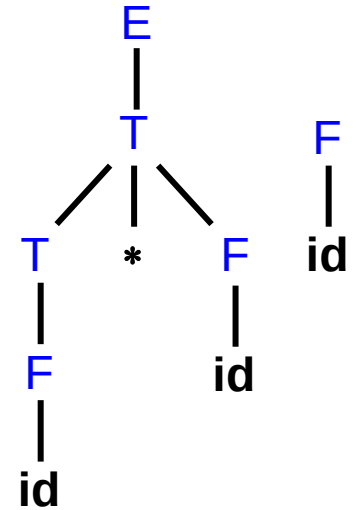
INPUT: id * id + id \$

STACK: 6
+
1
E
0

LR Parsing Program

State	action						goto		
	id	+	*	(	)	\$	E	T	F
0	s5			s4			1	2	3
1		s6				acc			
2		r2	s7		r2	r2			
3		r4	r4		r4	r4			
4	s5			s4			8	2	3
5		r6	r6		r6	r6			
6	s5			s4				9	3
7	s5			s4					10
8		s6			s11				
9		r1	s7		r1	r1			
10		r3	r3		r3	r3			
11		r5	r5		r5	r5			

OUTPUT:



GRAMMAR:

(1) $E \rightarrow E + T$

(2) $E' \rightarrow T$

(3) $T \rightarrow T * F$

(4) $T \rightarrow F$

(5) $F \rightarrow (E)$

(6) $F \rightarrow id$

LR Parser Example

INPUT:

id	*	id	+	id	\$
----	---	----	---	----	----

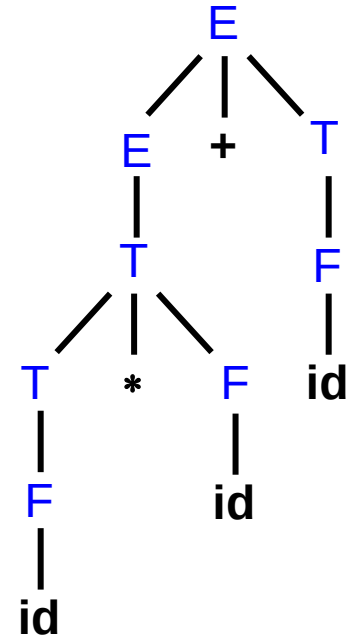
STACK:

9
T
6
+
1
E
0

LR Parsing Program

State	action						goto		
	id	+	*	(	)	\$	E	T	F
0	s5			s4			1	2	3
1		s6				acc			
2		r2	s7		r2	r2			
3		r4	r4		r4	r4			
4	s5			s4			8	2	3
5		r6	r6		r6	r6			
6	s5			s4				9	3
7	s5			s4					10
8		s6			s11				
9		r1	s7		r1	r1			
10		r3	r3		r3	r3			
11		r5	r5		r5	r5			

OUTPUT:



GRAMMAR:

(1) $E \rightarrow E + T$

(2) $E \rightarrow T$

(3) $T \rightarrow T * F$

(4) $T \rightarrow F$

(5) $F \rightarrow (E)$

(6) $F \rightarrow id$

LR Parser Example

INPUT:

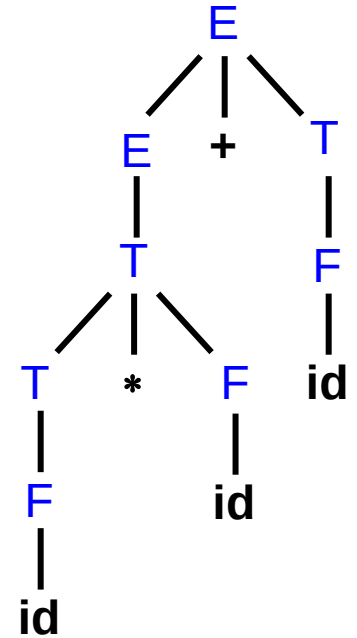
id * id + id \$

STACK:

0

LR Parsing Program

OUTPUT:



State	action						goto		
	id	+	*	(	)	\$	E	T	F
0	s5			s4			1	2	3
1		s6				acc			
2		r2	s7		r2	r2			
3		r4	r4		r4	r4			
4	s5			s4			8	2	3
5		r6	r6		r6	r6			
6	s5			s4				9	3
7	s5			s4					10
8		s6			s11				
9		r1	s7		r1	r1			
10		r3	r3		r3	r3			
11		r5	r5		r5	r5			

GRAMMAR:

- (1) $E \rightarrow E + T$
- (2) $E' \rightarrow T$
- (3) $T \rightarrow T * F$
- (4) $T \rightarrow F$
- (5) $F \rightarrow (E)$
- (6) $F \rightarrow id$

LR Parser Example

INPUT:

id * id + id \$

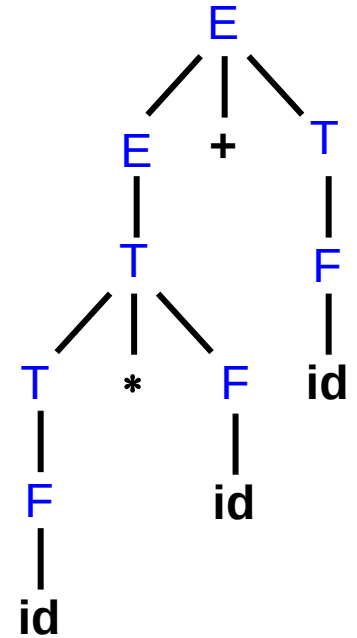
STACK:

1
E
0

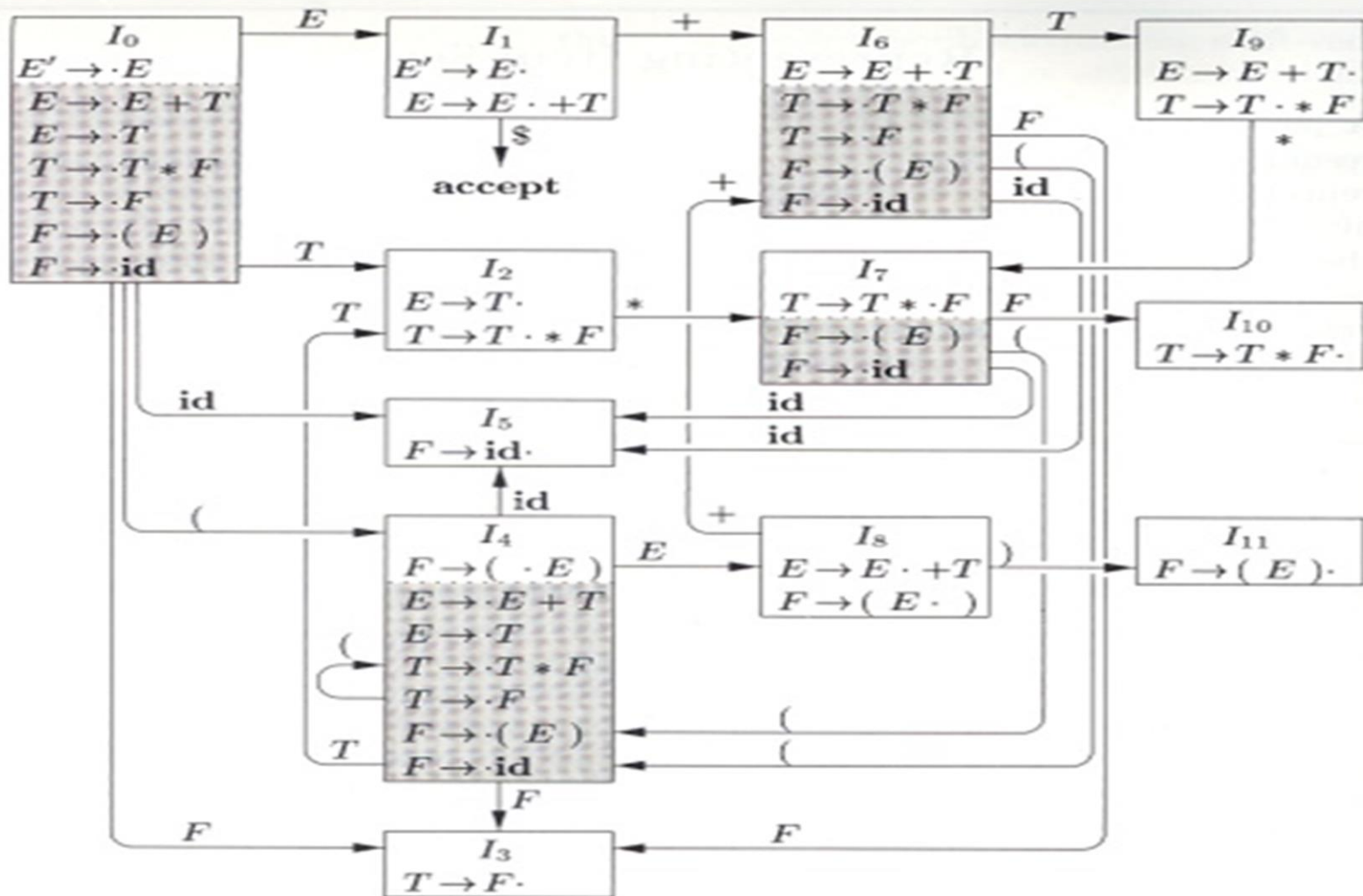
LR Parsing
Program

State	action						goto		
	id	+	*	(	)	\$	E	T	F
0	s5			s4			1	2	3
1		s6				acc			
2		r2	s7		r2	r2			
3		r4	r4		r4	r4			
4	s5			s4			8	2	3
5		r6	r6		r6	r6			
6	s5			s4				9	3
7	s5			s4					10
8		s6			s11				
9		r1	s7		r1	r1			
10		r3	r3		r3	r3			
11		r5	r5		r5	r5			

OUTPUT:



Conflicts in LR(0) Parsing Table



Another Example

Grammar:

1. $E \rightarrow E + T$

2. $E \rightarrow T$

3. $T \rightarrow T * F$

4. $T \rightarrow F$

5. $F \rightarrow (E)$

6. $F \rightarrow \text{id}$



state	action						goto		
	id	+	*	(	)	\$	E	T	F
0	s5			s4			1	2	3
1		s6				acc			
2	r2	r2	s7/r2	r2	r2	r2			
3	r4	r4	r4	r4	r4	r4			
4	s5			s4			8	2	3
5	r6	r6	r6	r6	r6	r6			
6	s5			s4				9	3
7	s5			s4					10
8		s6				s11			
9	r1	r1	s7/r1	r1	r1	r1			
10	r3	r3	r3	r3	r3	r3			
11	r5	r5	r5	r5	r5	r5			

shift as well as
reduce to do

shift-reduce conflict so not LR(0)

Conflicts LR(0) Parsing Table Contd.



- ✓ In the just obtained LR(0) parse table, shift and reduce occurs simultaneously in entry $[2, *]$ and $[9, *]$ (or it contains multiple-defined entries). This is called a shift reduce conflict.
- ✓ If the LR(0) parsing table contains any multiple-defined ACTION entries (or conflicts) then G is not LR(0)
- ✓ Two conflicts may arise during LR parsing:
 - shift-reduce: both shift and reduce possible in same item set
 - reduce-reduce: more than one distinct reduce action possible in same item set
- ✓ Conflicts can be resolved through look ahead in ACTION.
- ✓ In shift reduce conflict compiler gives more priority to shift than reduce.

Practice Problems



Consider the following Grammar G:

$$S \rightarrow AA$$

$$A \rightarrow aA$$

$$A \rightarrow b$$

Perform the LR(0) parsing of the input string **aabb.**

References

<https://www.javatpoint.com/lr-parser>

<https://www.geeksforgeeks.org/lr-parser/>

IMP Note to Self



Thank you